

图与网络分析

(Graph Theory and Network Analysis)

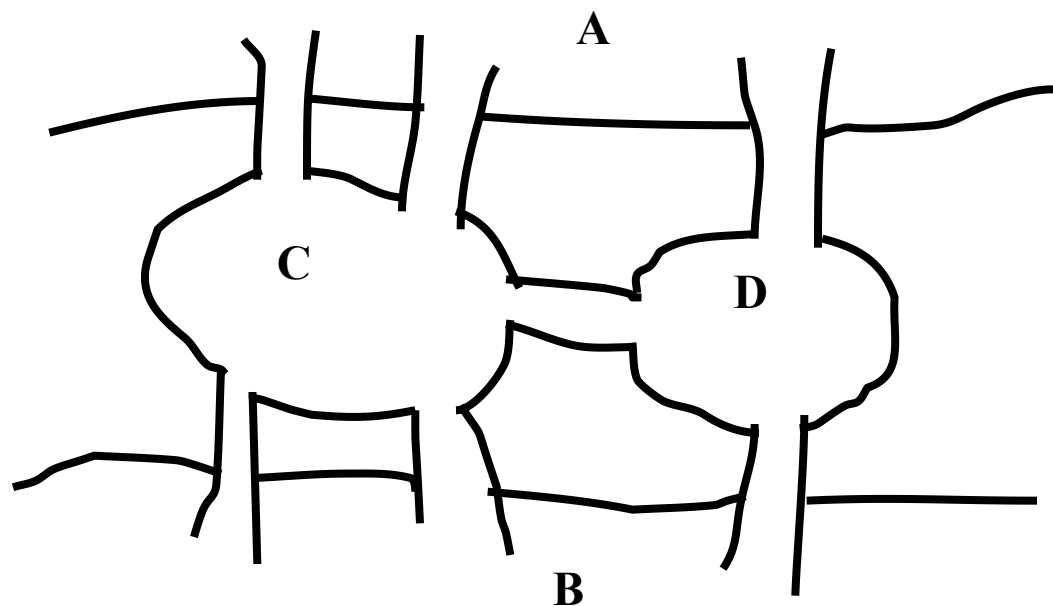
图与网络的基本知识

树及最小树问题

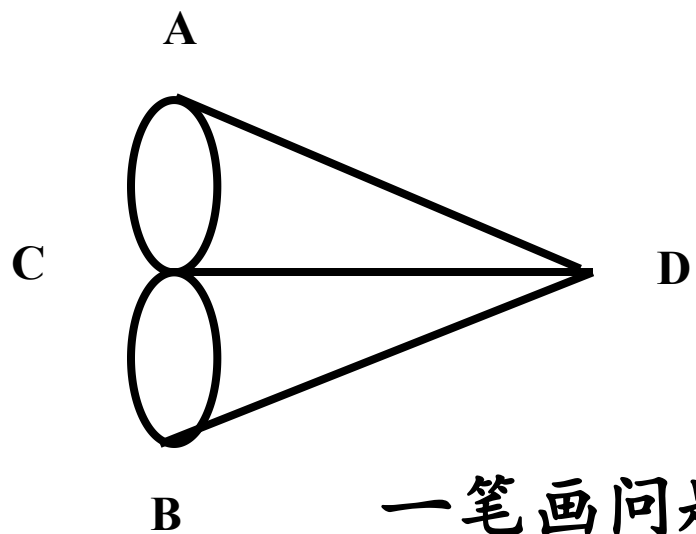
最短路问题

最大流问题

最小费用最大流问题



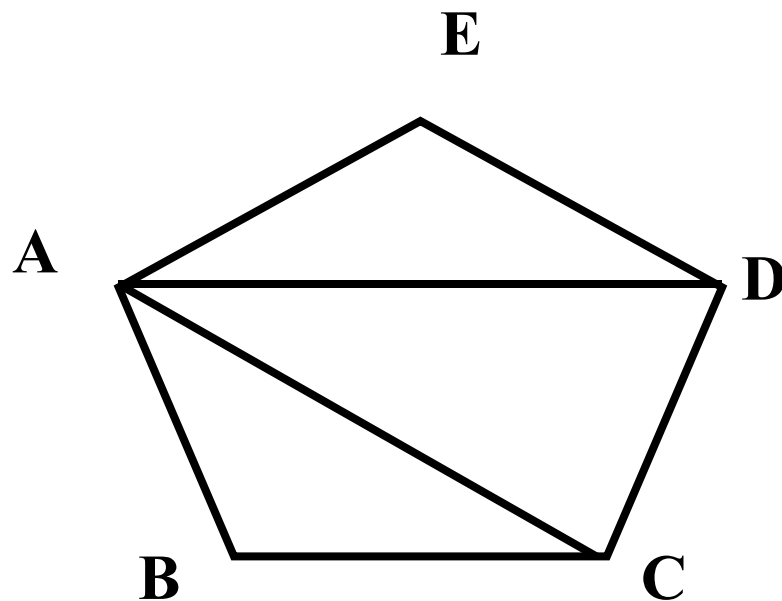
哥尼斯堡七空桥



一笔画问题

一、图与网络的基本知识

(一)、图与网络的基本概念



1、一个图是由点和连线组成。（连线可带箭头，也可不带，前者叫弧，后者叫边）

一个图是由点集 $V = \{v_j\}$ 和 V 中元素的无序对的一个集合 $E = \{e_k\}$ 构成的二元组，记为 $G = (V, E)$ ，其中 V 中的元素 v_j 叫做顶点， V 表示图 G 的点集合； E 中的元素 e_k 叫做边， E 表示图 G 的边集合。

例

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9, e_{10}\}$$

$$e_1 = \{v_1, v_2\} \quad e_2 = \{v_1, v_2\}$$

$$e_3 = \{v_2, v_3\} \quad e_4 = \{v_3, v_4\}$$

$$e_5 = \{v_1, v_3\} \quad e_6 = \{v_3, v_5\}$$

$$e_7 = \{v_3, v_5\} \quad e_8 = \{v_5, v_6\}$$

$$e_9 = \{v_6, v_6\} \quad e_{10} = \{v_1, v_6\}$$

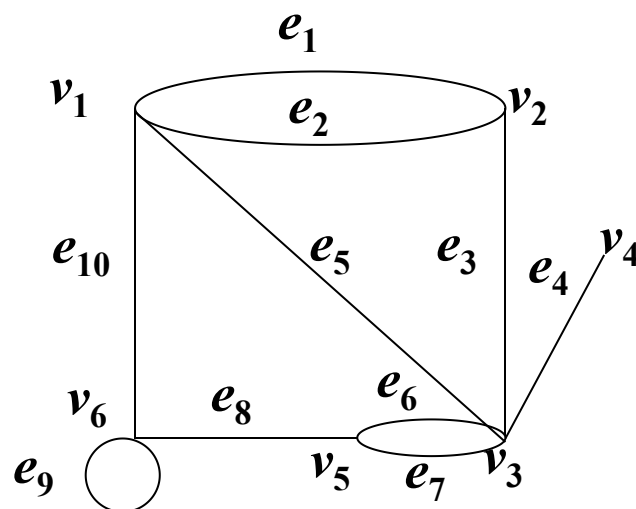


图1

2、如果一个图是由点和边所构成的，则称其为无向图，记作 $G = (V, E)$ ，连接点的边记作 $[v_i, v_j]$ ，或者 $[v_j, v_i]$ 。

3、如果一个图是由点和弧所构成的，那么称它为有向图，记作 $D = (V, A)$ ，其中 V 表示有向图 D 的点集合， A 表示有向图 D 的弧集合。一条方向从 v_i 指向 v_j 的弧，记作 (v_i, v_j) 。

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6\},$$

$$A = \{(v_1, v_3), (v_2, v_1), (v_2, v_3),$$

$$(v_2, v_5), (v_3, v_5), (v_4, v_5),$$

$$(v_5, v_4), (v_5, v_6)\}$$

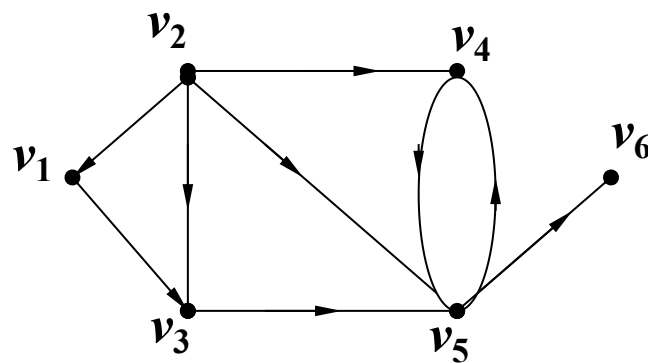


图2

4、一条边的两个端点是相同的,那么称为这条边是环。

5、如果两个端点之间有两条以上的边,那么称为它们为多重边。

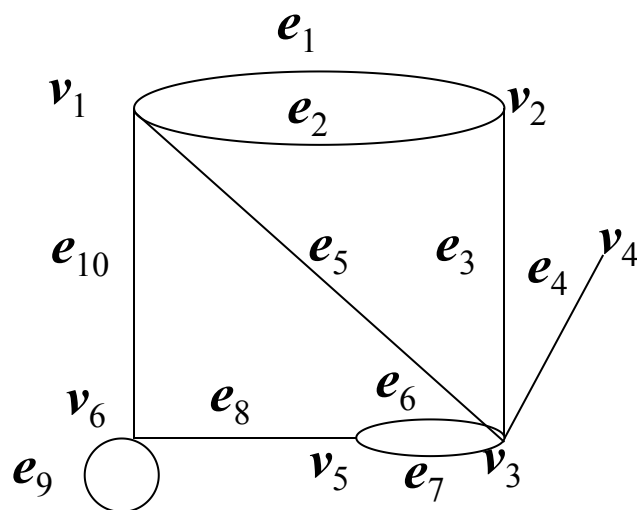
6、一个无环,无多重边的图称为简单图,一个无环,有多重边的图称为多重图。

7、每一对顶点间都有边相连的无向简单图称为完全图。

有向完全图则是指任意两个顶点之间有且仅有一条有向边的简单图。

8、以点 v 为端点的边的个数称为点 v 的度（次），记作 $d(v)$ 。 图中 $d(v_1)=4$, $d(v_6)=4$ （环计两度）

度为零的点称为孤立点，度为1的点称为悬挂点。悬挂点的关联边称为悬挂边。度为奇数的点称为奇点，度为偶数的点称为偶点。



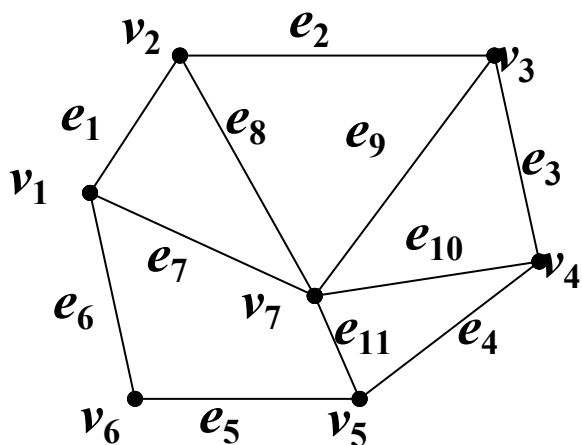
定理1 所有顶点度数之和等于所有边数的2倍。

定理2 在任一图中，奇点的个数必为偶数。

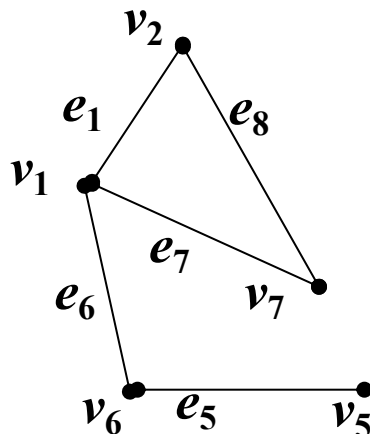
有向图中，以 v_i 为始点的边数称为点 v_i 的出次，用表示 $d^+(v_i)$ ；以 v_i 为终点的边数称为点 v_i 的入次，用 $d^-(v_i)$ 表示； v_i 点的出次和入次之和就是该点的次。

所有顶点的入次之和等于所有顶点的出次之和。

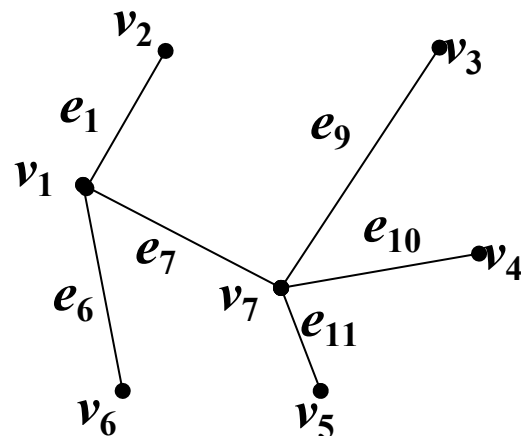
9、设 $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$ 如果 $V_2 \subseteq V_1$, $E_2 \subseteq E_1$ 称 G_2 是 G_1 的子图；如果 $V_2 = V_1$, $E_2 \subseteq E_1$ 称 G_2 是 G_1 的部分图或支撑子图。



(a)



(b)



(c)

子图

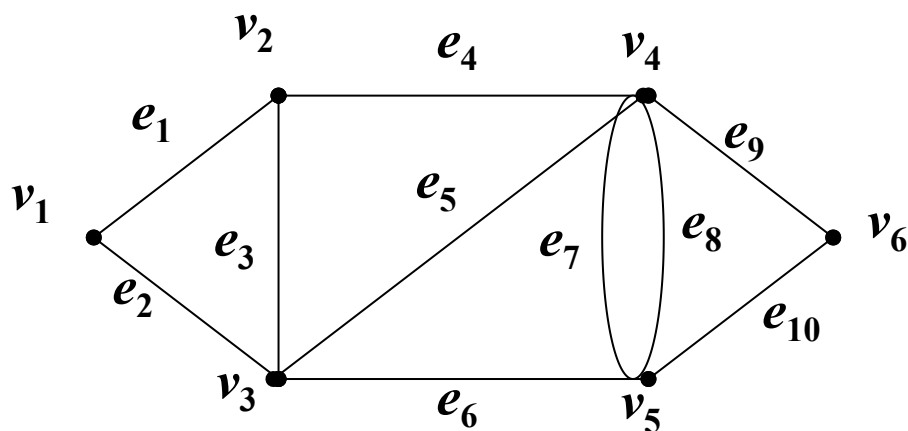
支撑子图

在实际应用中，给定一个图 $G = (V, E)$ 或有向图 $D = (V, A)$ ，在 V 中指定两个点，一个称为始点（或发点），记作 v_1 ，一个称为终点（或收点），记作 v_n ，其余的点称为中间点。对每一条弧 $(v_i, v_j) \in A$ ，对应一个数 w_{ij} 称为弧上的“权”。通常把这种赋权的图称为网络。

10、由两两相邻的点及其相关联的边构成的点边序列称为链。

如： $v_0, e_1, v_1, e_2, v_2, e_3, v_3, \dots, v_{n-1}, e_n, v_n$ ，
记作 $(v_0, v_1, v_2, v_3, \dots, v_{n-1}, v_n)$ ，

其链长为 n ，其中 v_0 ， v_n 分别称为链的起点和终点。若点边序列中没有重复点和重复边者为初等链。



11、图中任意两点之间均至少有一条通路，则称此图为连通图，否则称为不连通图。

(二)、图的矩阵表示

对于网络（赋权图） $G = (V, E)$ ，其中边 (v_i, v_j) 有权 w_{ij} ，构造矩阵 $A = (a_{ij})_{n \times n}$ ，其中：

$$a_{ij} = \begin{cases} w_{ij} & (v_i, v_j) \in E \\ 0 & (v_i, v_j) \notin E \end{cases}$$

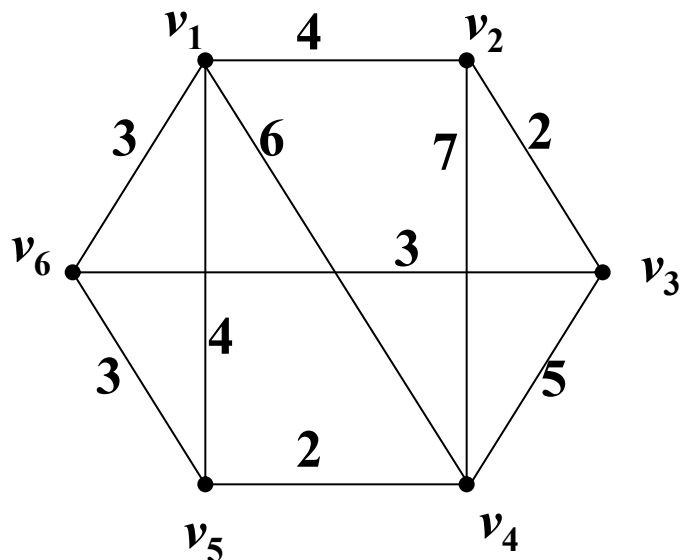
称矩阵A为网络G的权矩阵。

设图 $G = (V, E)$ 中顶点的个数为n，构造一个矩阵 $A = (a_{ij})_{n \times n}$ ，其中：

$$a_{ij} = \begin{cases} 1 & (v_i, v_j) \in E \\ 0 & (v_i, v_j) \notin E \end{cases}$$

称矩阵A为网络G的邻接矩阵。

例



权矩阵为：

$$A = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 & v_4 & v_5 & v_6 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} & \begin{bmatrix} 0 & 4 & 0 & 6 & 4 & 3 \\ 4 & 0 & 2 & 7 & 0 & 0 \\ 0 & 2 & 0 & 5 & 0 & 3 \\ 6 & 7 & 5 & 0 & 2 & 0 \\ 4 & 0 & 0 & 2 & 0 & 3 \\ 3 & 0 & 3 & 0 & 3 & 0 \end{bmatrix} \end{matrix}$$

邻接矩阵为：

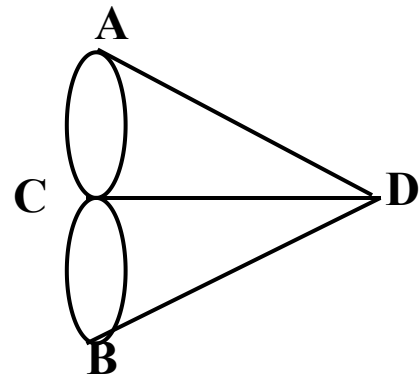
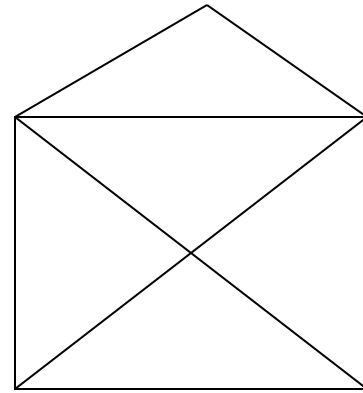
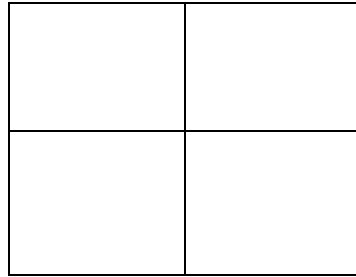
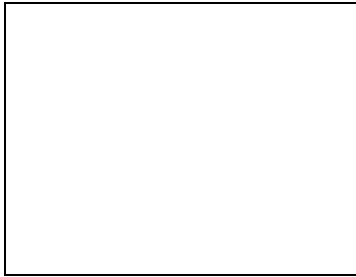
$$B = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 & v_4 & v_5 & v_6 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

(三)、 欧拉回路与中国邮路问题

定义8.18 连通图 G 中，若存在一条道路，经过每边一次且仅一次，则称这条路为欧拉道路。若存在一条回路，经过每边一次且仅一次，则称这条回路为欧拉回路。

具有欧拉回路的图称为欧拉图。

定理8.7 无向连通图 G 是欧拉图，当且仅当 G 中无奇点。



奇偶点图上作业法

(1) 找出图 G 中的所有的奇顶点，把它们两两配成对，而每对奇点之间必有一条通路，把这条通路上的所有边作为重复边追加到图中去，这样得到的新连通图必无奇点。

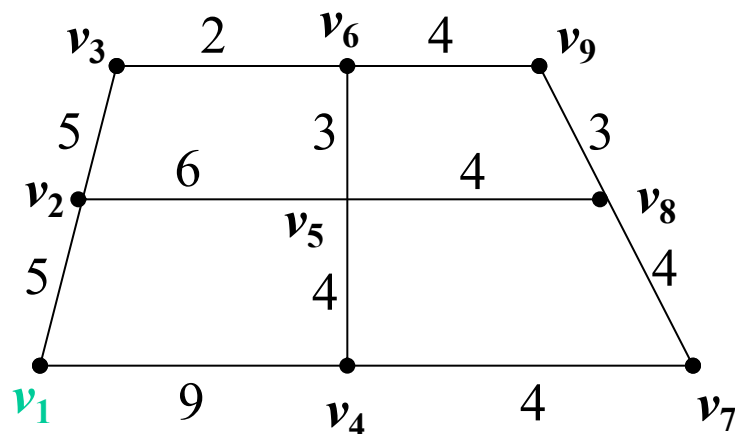
(2) 如果边 $e = (u, v)$ 上的重复边多于一条，则可从重复边中去掉偶数条，使得其重复边至多为一条，图中的顶点仍全部都是偶顶点。

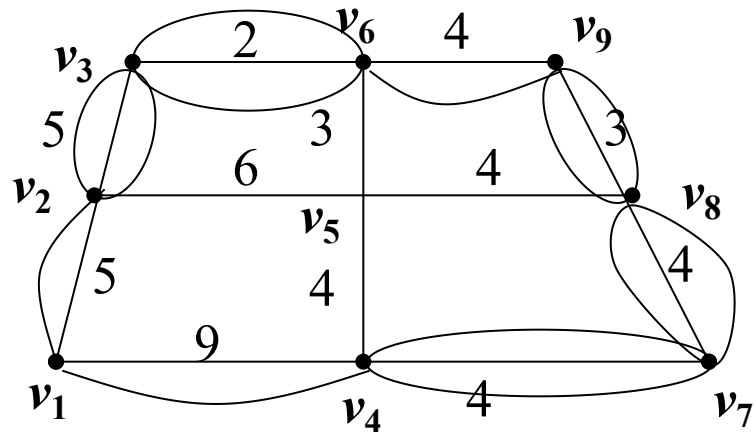
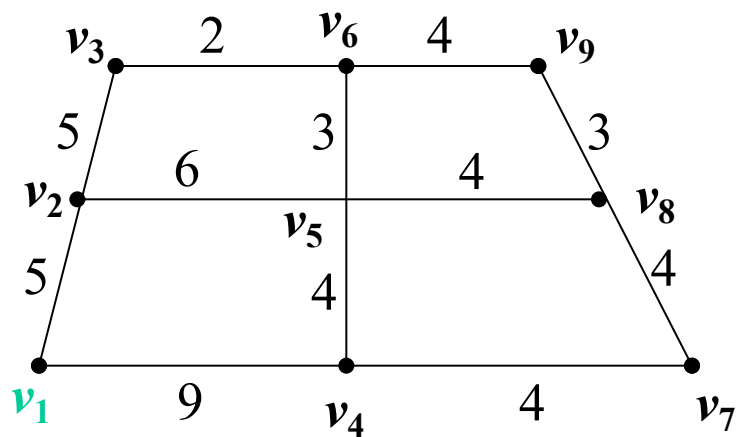
(3) 检查图中的每一个圈，如果每一个圈的重复边的总长不大于该圈总长的一半，则已经求得最优方案。如果存在一个圈，重复边的总长大于该圈总长的一半时，则将这个圈中的重复边去掉，再将该圈中原来没有重复边的各边加上重复边，其它各圈的边不变，返回步骤(2)。

判定标准1：在最优邮递路线上，图中的每一条边至多有一条重复边。

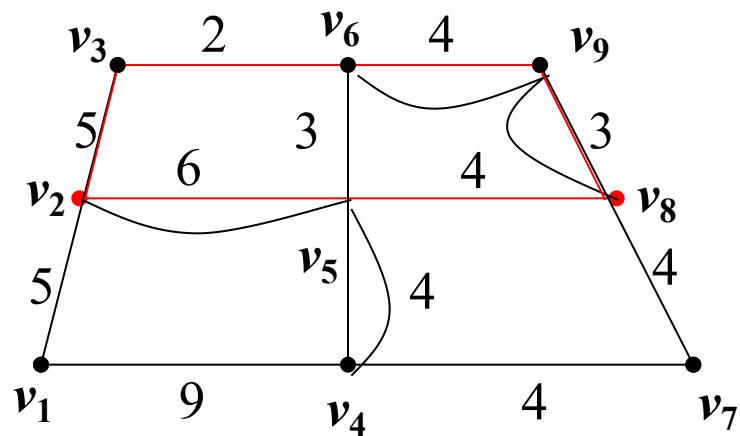
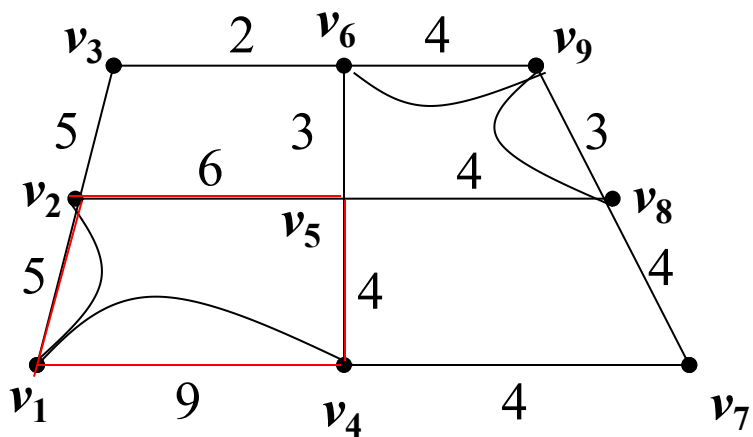
判定标准2：在最优邮递路线上，图中每一个圈的重复边总权小于或等于该圈总权的一半。

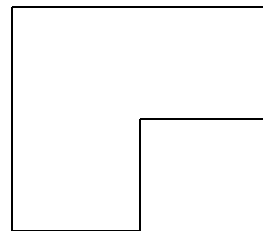
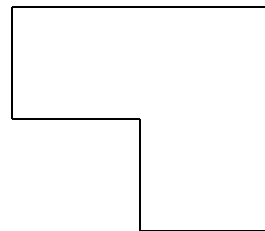
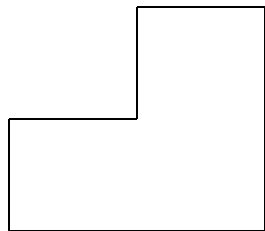
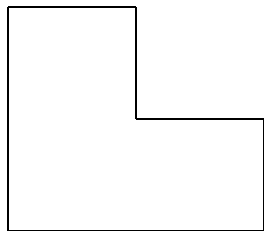
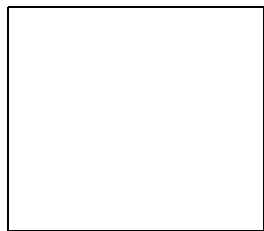
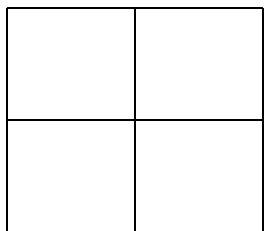
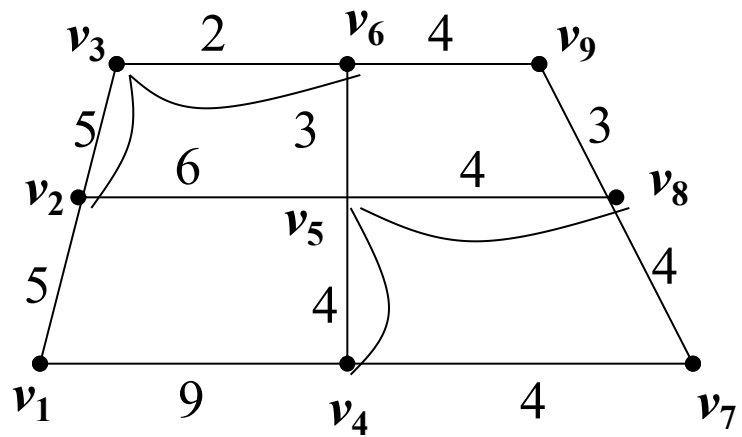
例 求解下图所示网络的中国邮路问题，图中数字为该边的长。





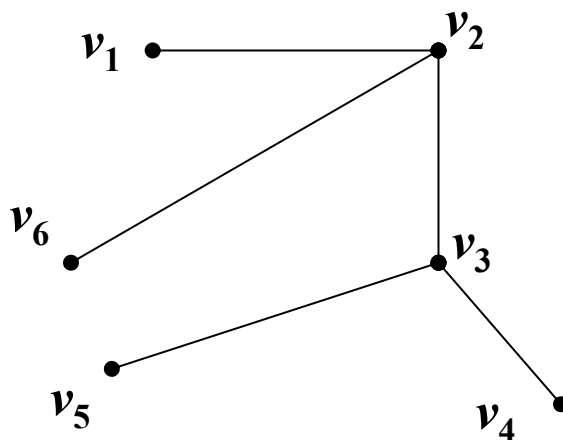
$$l_{12}+2 l_{23}+2 l_{36}+2 l_{89}+2 l_{78}+l_{69}+l_{14}+2 l_{47}=51$$





二、 树及最小树问题

已知有六个城市，它们之间要架设电话线，要求任意两个城市均可以互相通话，并且电话线的总长度最短。



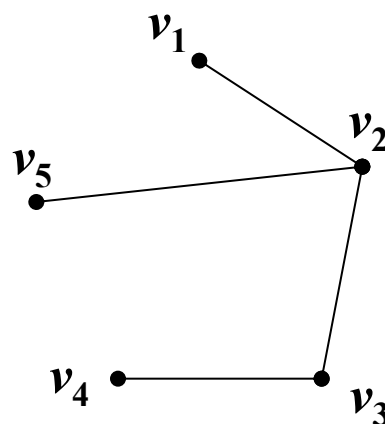
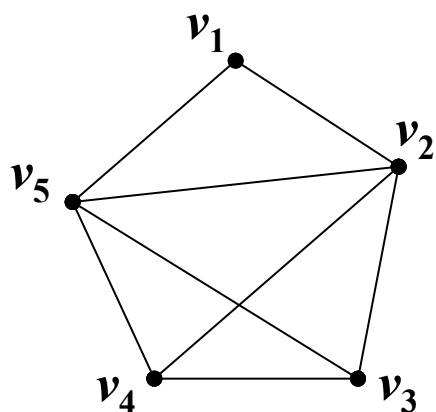
定义 14 连通且不含圈的无向图称为树。树中次为 1 的点称为树叶,次大于 1 的点称为分枝点。

下面研究树的性质。树的性质可用下面定理表出。

定理 6 图 $T=(V,E)$, $|V|=n$, $|E|=m$, 则下列关于树的说法是等价的。

- (1) T 是一个树。
- (2) T 无圈, 且 $m=n-1$ 。
- (3) T 连通, 且 $m=n-1$ 。
- (4) T 无圈, 但每加一新边即得唯一的一个圈。
- (5) T 连通, 但每舍去一边就不连通。
- (6) T 中任意两点, 有唯一链相连。

定义15 设图 $K=(V, E_1)$ 是图 $G=(V, E)$ 的一支撑子图, 如果图 $K=(V, E_1)$ 是一个树, 那么称 K 是 G 的一个生成树 (支撑树), 或简称为图 G 的树。图 G 中属于生成树的边称为树枝, 不在生成树中的边称为弦。



一个图 G 有生成树的充要条件是 G 是连通图。

连通图的生成树的方法：

1 避圈法

2 破圈法

按照边的选法不同,找图中生成树的方法可分为两种:

(1) 深探法

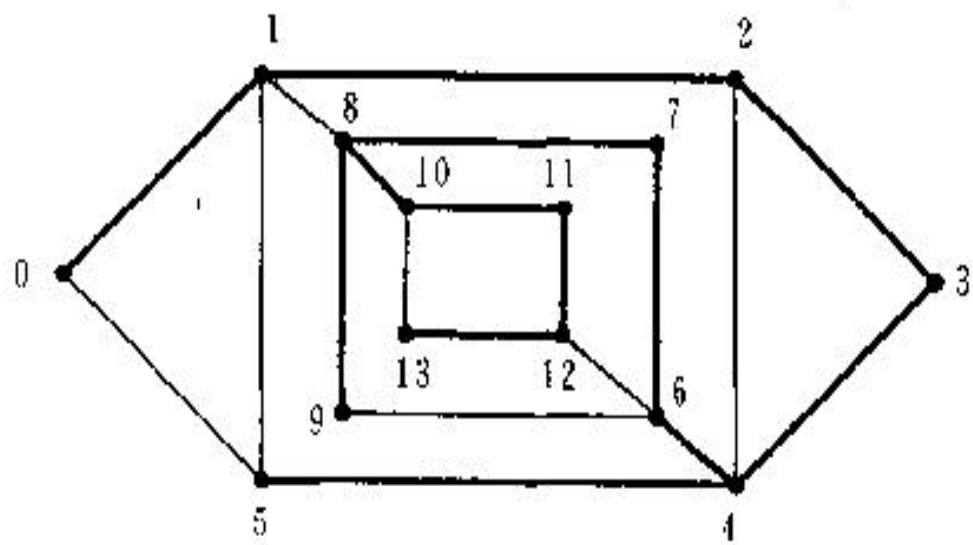
步骤如下:(用标号法)

① 在点集 V 中任取一点 v , 给 v 以标号 0。

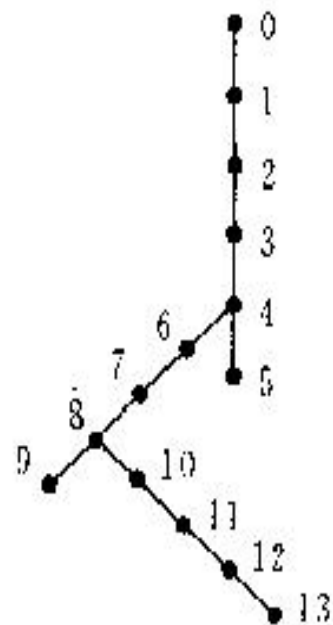
② 若某点 u 已得标号 i , 检查一端点为 u 的各边, 另一端点是否均已标号。

若有 (u, w) 边之 w 未标号, 则给 w 以标号 $i+1$, 记下边 (u, w) 。令 w 代 u , 重复②。

若这样的边的另一端点均已有标号, 就退到标号为 $i-1$ 的 r 点, 以 r 代 u , 重复②。直到全部点得到标号为止。



(a)



(b)

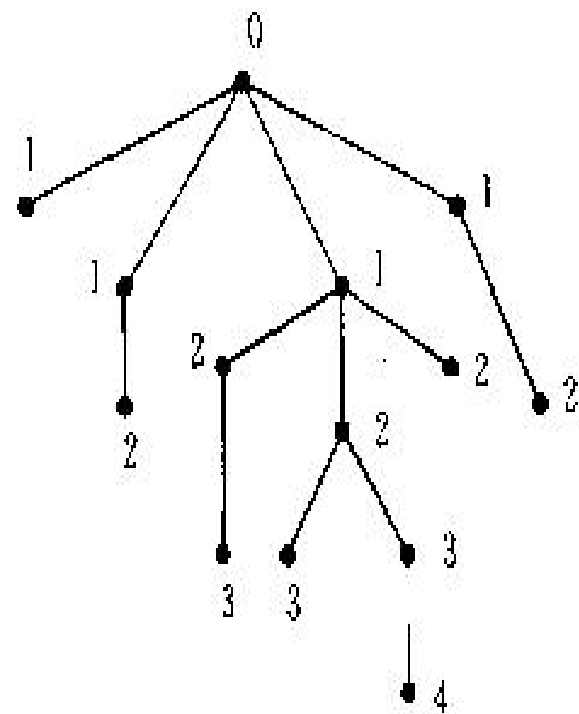
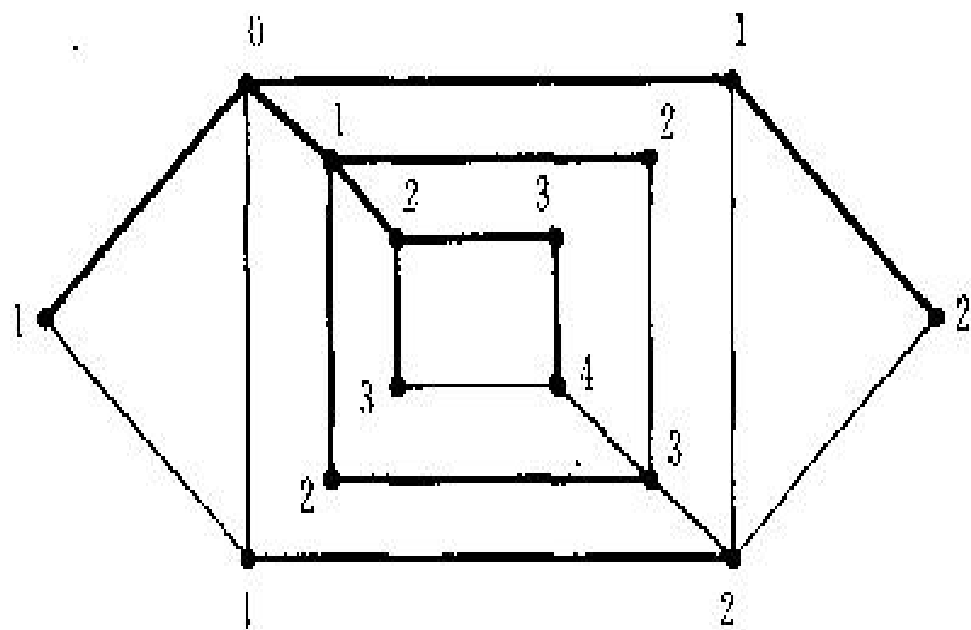
(2) 广探法

步骤如下：

① 在点集 V 中任取一点 v , 给 v 以标号 0。

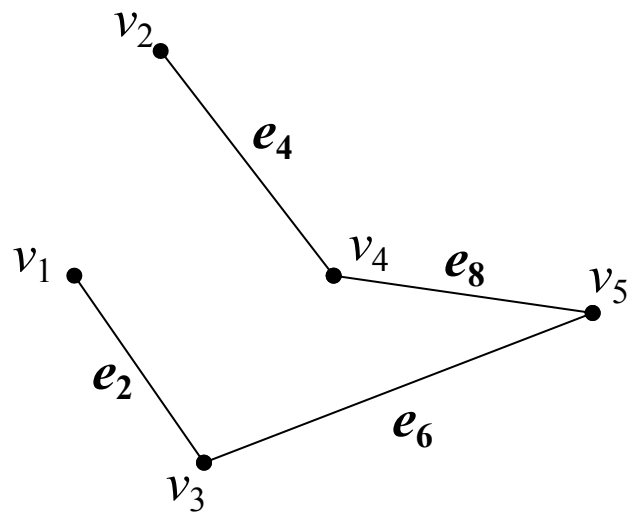
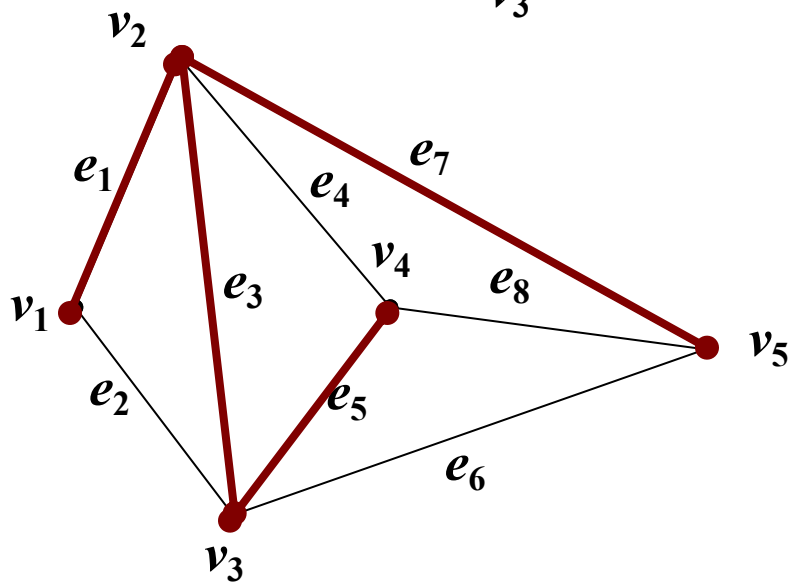
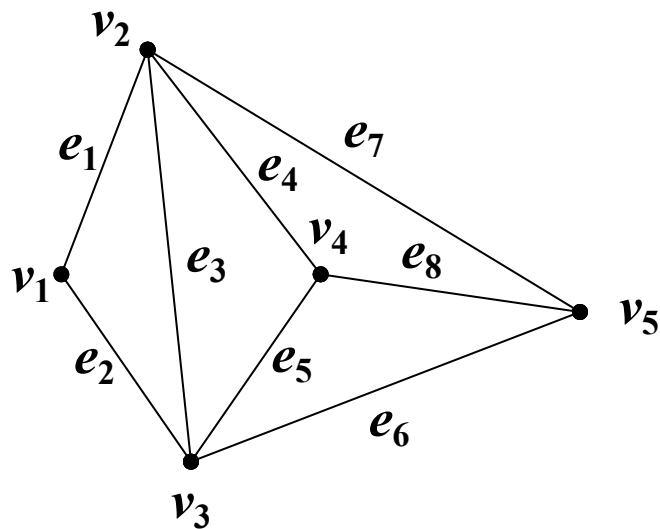
② 令所有标号为 i 的点集为 V_i , 检查 $[V_i, V \setminus V_i]$ 中的边端点是否均已标号。对所有未标号之点均标以 $i+1$, 记下这些边。

③ 对标号 $i+1$ 的点重复步骤②, 直到全部点得到标号为止。



相对于避圈法,还有一种求生成树的方法叫破圈法。这种方法是在图 G 中任意取一个圈,从圈上任意舍弃一条边,将这个圈破掉。重复这个步骤直到图 G 中没有圈为止。

用破圈法求出下图的一个生成树。



三 最小生成树问题

如果图 $T=(V, E_1)$ 是图 G 的一个生成树，那么称 E_1 上所有边的权的和为生成树 T 的权，记作 $S(T)$ 。如果图 G 的生成树 T^* 的权 $S(T^*)$ ，在 G 的所有生成树 T 中的权最小，即

$$S(T^*) = \min_T S(T)$$

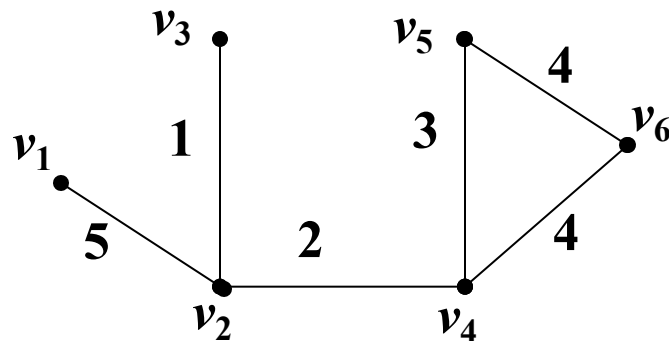
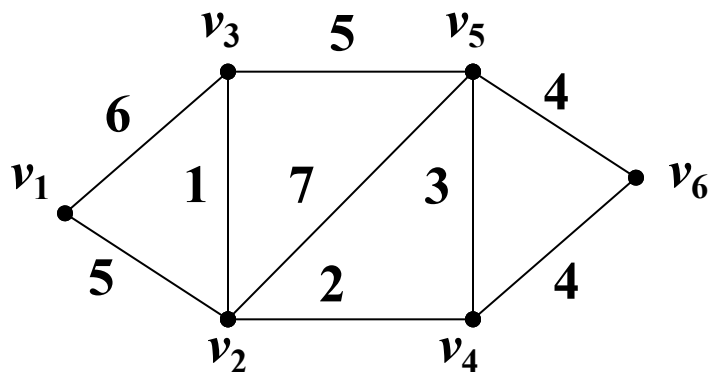
那么称 T^* 是 G 的最小生成树。

许多网络问题都可以归结为最小树问题。例如，设计长度最小的公路网把若干城市联系起来；设计用料最省的电话网把有关单位联系起来等。

算法 1 Kruskal算法

每步从未选边中选取边 e ，使它与已选边不构成圈，且 e 是未选边中的最小权边，直到选够 $n-1$ 条边为止。

例 某六个城市之间的道路网如图 所示，要求沿着已知长度的道路联结六个城市的电话线网，使电话线的总长度最短。



算法 2 破圈法

基本步骤:

(1) 从图 G 中任选一棵树 T_1 。

(2) 加上一条弦 e_1 , $T_1 + e_1$ 中立即生成一个圈。去掉此圈中最大权边, 得到新树 T_2 。以 T_2 代 T_1 , 重复(2)再检查剩余的弦, 直到全部弦检查完毕为止。

四、根树及其应用

前面几节我们讨论的树都是无向树,本节讨论有向树。有向树中的根树在计算机科学、决策论中有重要应用。

定义 17 若一个有向图在不考虑边的方向时是一棵树,则称这个有向图为有向树。

定义 18 有向树 T , 恰有一个结点入次为 0, 其余各点入次均为 1, 则称 T 为根树(又称外向树)。

根树中入次为 0 的点称为根。根树中出次为 0 的点称为叶, 其它顶点称为分枝点。由根到某一顶点 v_i 的道路长度(设每边长度为 1), 称为 v_i 点的层次。

如图 8-29 所示的树是根树, 其中 v_1 为根, v_1, v_2, v_3, v_4, v_8 为分枝点, 其余各点为叶, 顶点 v_2, v_3, v_4 的层次为 1, 顶点 v_{11} 的层次为 3 等等。

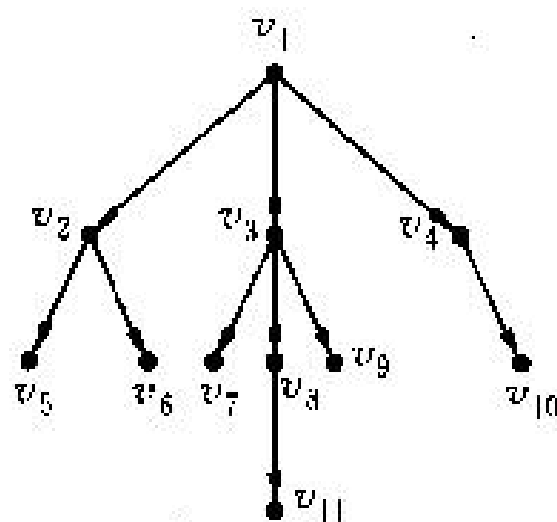


图 8-29

定义 19 在根树中,若每个顶点的出次小于或等于 m ,称这棵树为 m 叉树。若每个顶点的出次恰好等于 m 或零,则称这棵树为完全 m 叉树。当 $m=2$ 时,称为二叉树、完全二叉树。

例如图 8-30 中(a)为完全三叉树、(b)为四叉树。

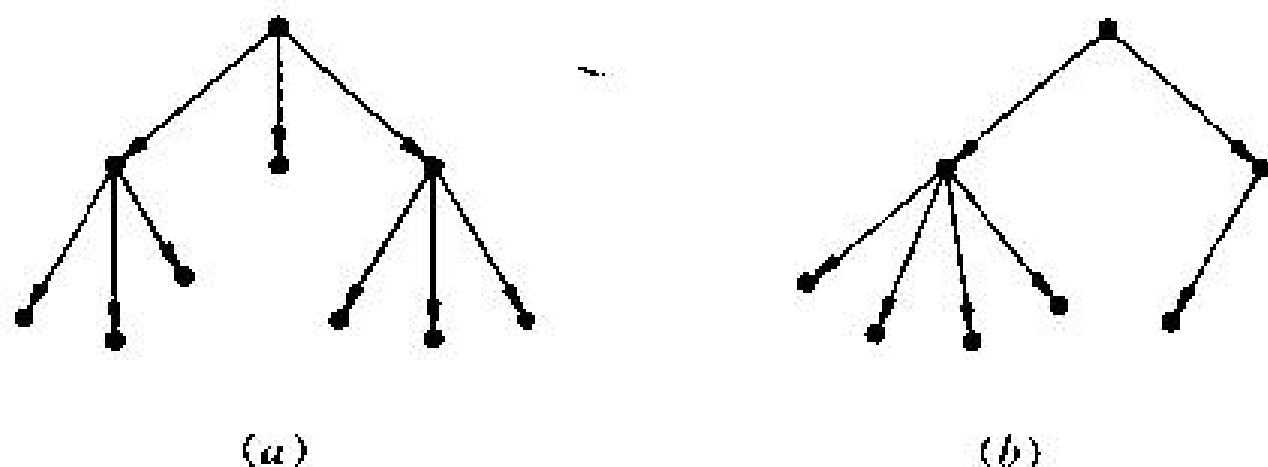


图 8-30

在实际问题中常讨论叶子上带权的二叉树。令有 s 个叶子的二叉树 T 各叶子的权分别为 p_i , 根到各叶子的距离(层次)为 $l_i (i=1, \dots, s)$, 这样二叉树 T 的总权数

$$m(T) = \sum_{i=1}^s p_i l_i$$

满足总权最小的二叉树称为最优二叉树。霍夫曼(D A Huffman)给出了一个求最优二叉树的算法, 所以又称霍夫曼树。

算法步骤为:

- (1) 将 s 个叶子按权由小至大排序, 不失一般性, 设 $p_1 \leq p_2 \leq \dots \leq p_s$ 。
- (2) 将二个具有最小权的叶子合并成一个分枝点, 其权为 $p_1 + p_2$, 将新的分枝点作为一个叶子。令 $s \leftarrow s-1$, 若 $s=1$ 停止, 否则转(1)。

例 9 $s=6$, 其权分别为 4, 3, 3, 2, 2, 1, 求最优二叉树。

解: 该树构造过程见图 8-31。总权为

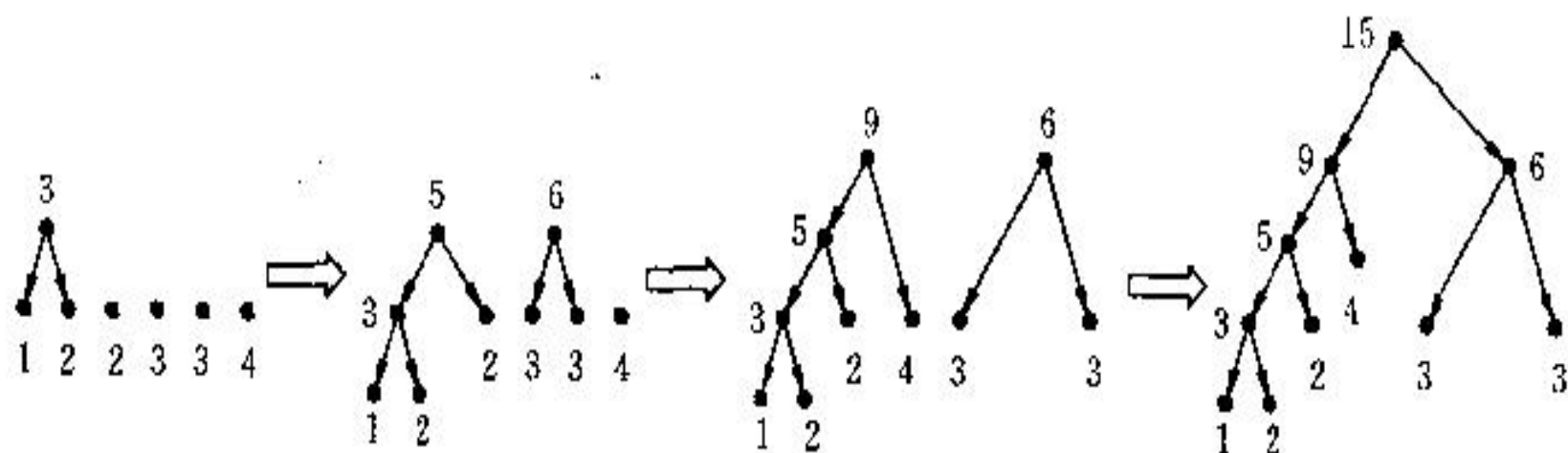


图 8-31

$$1 \times 4 + 2 \times 4 + 2 \times 3 + 3 \times 2 + 3 \times 2 + 4 \times 2 = 38$$

可以证明此算法得到的树为最优二叉树, 直观意义: 叶子的距离是依权的递减而增加, 所以总权最小。

三、最短路问题

最短路的一般提法为：设 $G = (V, E)$ 为连通图，图中各边 (v_i, v_j) 有权 l_{ij} ($l_{ij} = +\infty$ 表示 v_i, v_j 之间没有边)， v_s, v_t 为图中任意两点，求一条路 μ ，使它为从 v_s 到 v_t 的所有路中总权最短。即：

$$L(\mu) = \sum_{(v_i, v_j) \in \mu} l_{ij} \quad \text{最小。}$$

三、最短路问题

本算法由 Dijkstra 于 1959 年提出,可用于求解指定两点 v_s, v_t 间的最短路,或从指定点 v_s 到其余各点的最短路,目前被认为是求无负权网络最短路问题的最好方法。算法的基本思路基于以下原理:若序列 $\{v_s, v_1, \dots, v_{n-1}, v_n\}$ 是从 v_s 到 v_n 的最短路,则序列 $\{v_s, v_1, \dots, v_{n-1}\}$ 必为从 v_s 到 v_{n-1} 的最短路。

下面给出 Dijkstra 算法基本步骤,采用标号法。可用两种标号: T 标号与 P 标号, T 标号为试探性标号(tentative label), P 为永久性标号(permanent label),给 v_i 点一个 P 标号时,表示从 v_s 到 v_i 点的最短路权, v_i 点的标号不再改变。给 v_i 点一个 T 标号时,表示从 v_s 到 v_i 点的估计最短路权的上界,是一种临时标号,凡没有得到 P 标号的点都有 T 标号。算法每一步都把某一点的 T 标号改为 P 标号,当终点 v_t 得到 P 标号时,全部计算结束。对于有 n 个顶点的图,最多经 $n-1$ 步就可以得到从始点到终点的最短路。

(一)、 Dijkstra算法

适用于 $w_{ij} \geq 0$ ，给出了从 v_s 到任意一个点 v_j 的最短路。

算法步骤:

1.给始点 v_s 以P标号 $P(v_s) = 0$ ，这表示从 v_s 到 v_s 的最短距离为0，其余节点均给T标号， $T(v_i) = +\infty$ ($i = 2, 3, \dots, n$)。

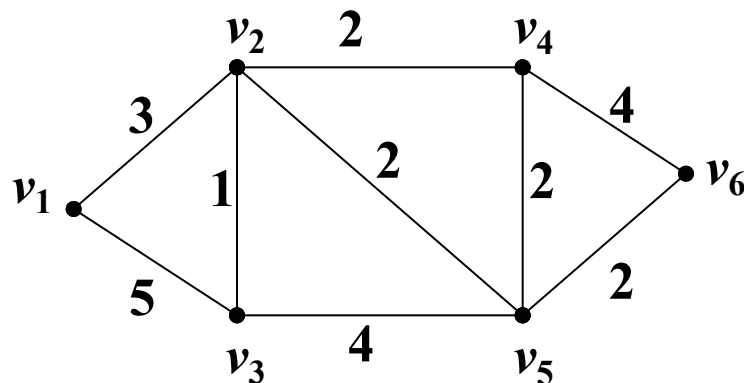
2.设节点 v_i 为刚得到P标号的点，考虑点 v_j ，其中
 $(v_i, v_j) \in E$ ，且 v_j 为T标号。对 v_j 的T标号进行如下修改：
$$T(v_j) = \min[T(v_j), P(v_i) + l_{ij}]$$

3.比较所有具有T标号的节点，把最小者改为P标号，即：

$$P(v_k) = \min[T(v_i)]$$

当存在两个以上最小者时，可同时改为P标号。若全部节点均为P标号，则停止，否则用 v_k 代替 v_i ，返回步骤 (2)。

例一、 用Dijkstra算法求下图从 v_1 到 v_6 的最短路。



解 (1) 首先给 v_1 以P标号, 给其余所有点T标号。

$$P(v_1) = 0 \quad T(v_i) = +\infty \quad (i = 2, 3, \dots, 6)$$

$$(2) \quad T(v_2) = \min[T(v_2), P(v_1) + l_{12}] = \min[+\infty, 0 + 3] = 3$$

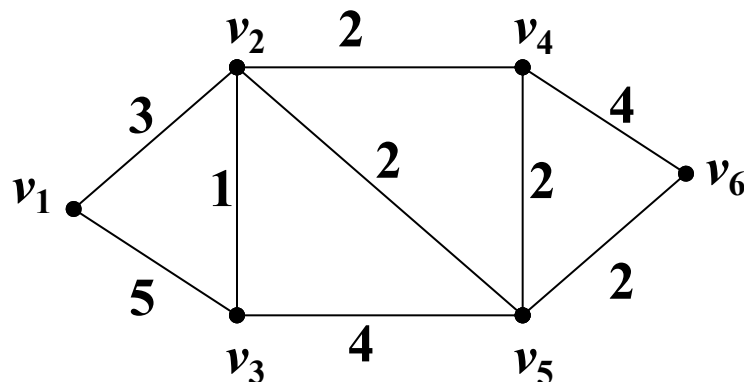
$$T(v_3) = \min[T(v_3), P(v_1) + l_{13}] = \min[+\infty, 0 + 5] = 5$$

$$(3) \quad P(v_2) = 3$$

$$(4) \quad T(v_3) = \min[T(v_3), P(v_2) + l_{23}] = \min[5, 3 + 1] = 4$$

$$T(v_4) = \min[T(v_4), P(v_2) + l_{24}] = \min[+\infty, 3 + 2] = 5$$

$$T(v_5) = \min[T(v_5), P(v_2) + l_{25}] = \min[+\infty, 3 + 2] = 5$$



$$(5) \quad P(v_3) = 4$$

$$(6) \quad T(v_5) = \min[T(v_5), P(v_3) + l_{35}] = \min[6, 4 + 4] = 8$$

$$(7) \quad P(v_4) = 5 \quad P(v_5) = 5$$

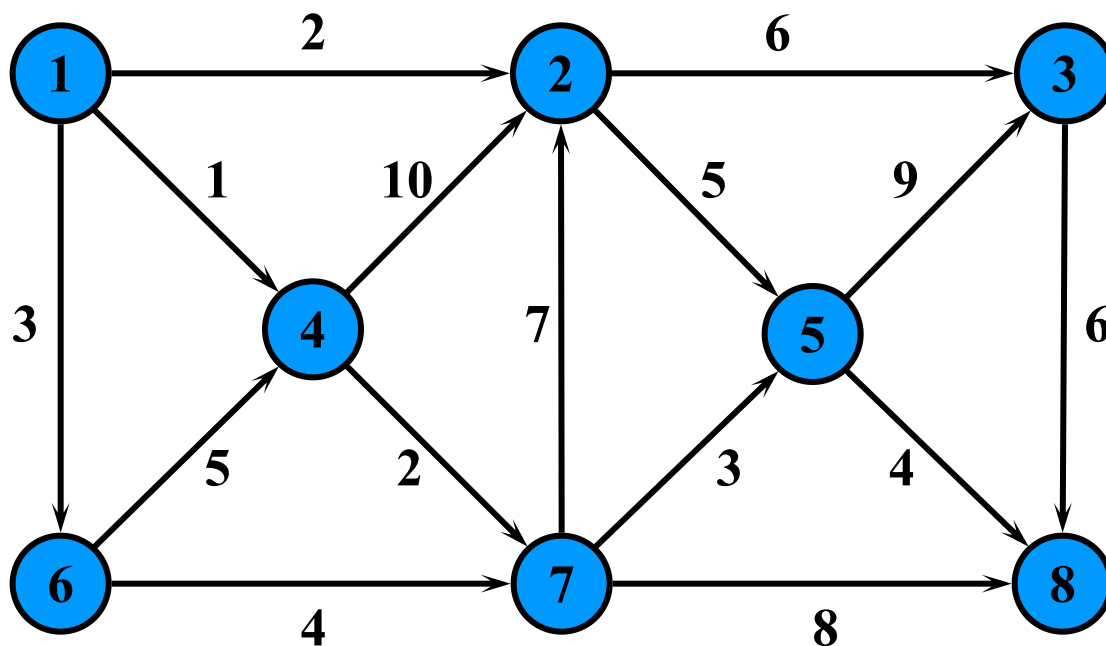
$$(8) \quad T(v_6) = \min[T(v_6), P(v_4) + l_{46}] = \min[+\infty, 5 + 4] = 9$$

$$(9) \quad T(v_6) = \min[T(v_6), P(v_5) + l_{56}] = \min[+\infty, 5 + 2] = 7$$

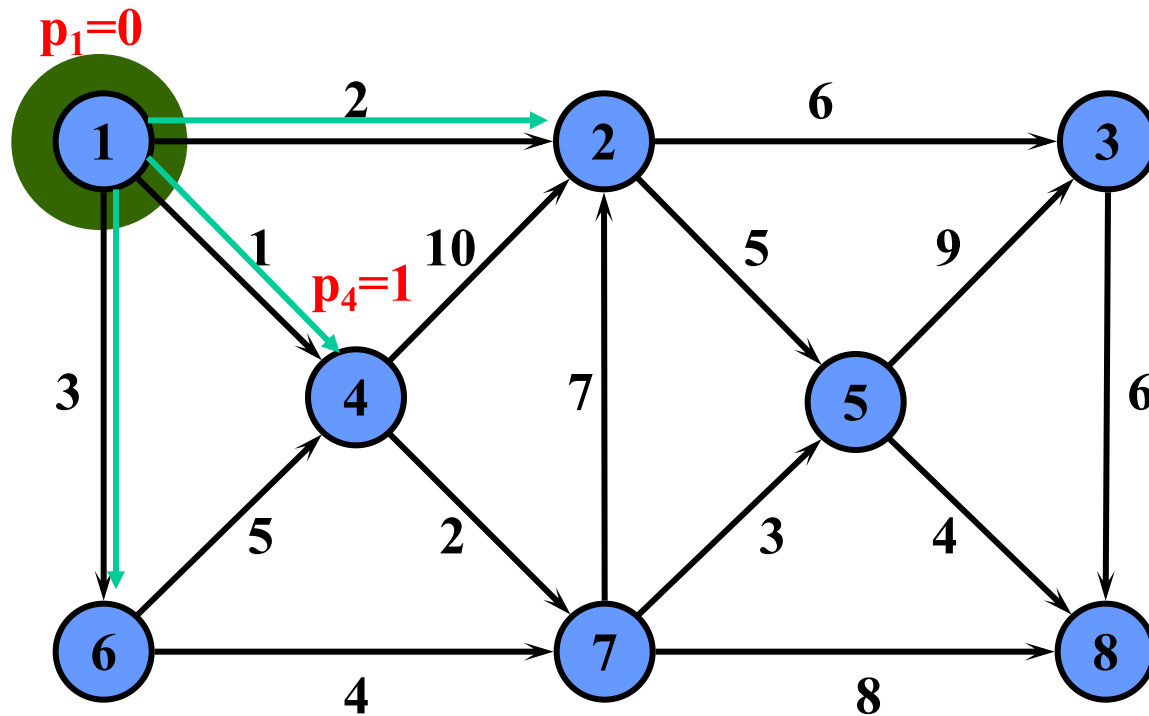
$$(10) \quad P(v_6) = 7$$

反向追踪得 v_1 到 v_6 的最短路为: $v_1 \rightarrow v_2 \rightarrow v_5 \rightarrow v_6$

求从1到8的最短路径



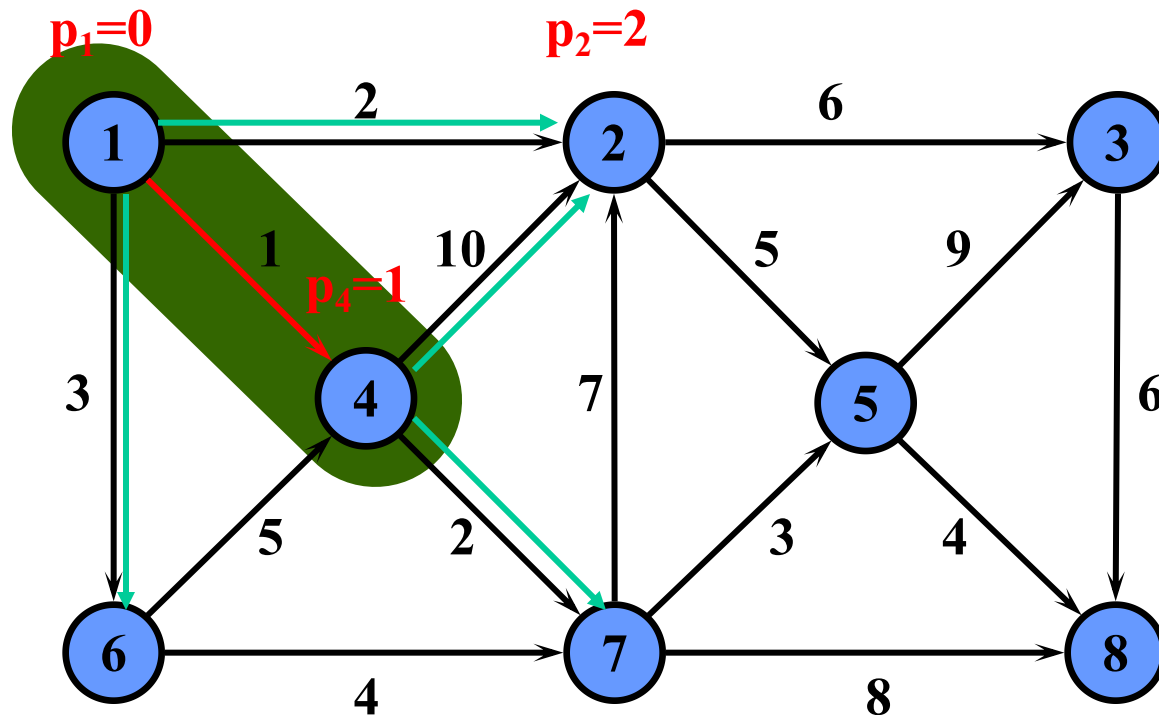
$$X=\{1\}, w_1=0$$



$$\min \{c_{12}, c_{14}, c_{16}\} = \min \{0+2, 0+1, 0+3\} = \min \{2, 1, 3\} = 1$$

$$X=\{1,4\}, p_4=1$$

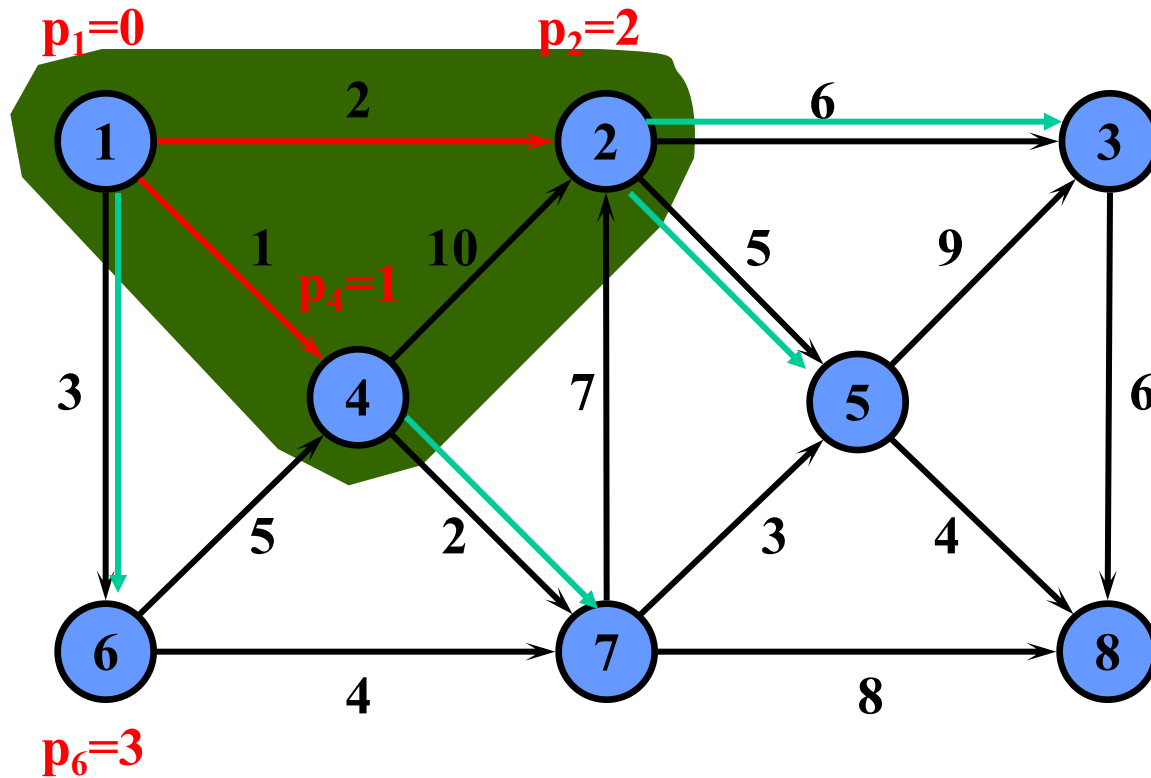
$$X=\{1,4\}$$



$$\min \{c_{12}, c_{16}, c_{42}, c_{47}\} = \min \{0+2, 0+3, 1+10, 1+2\} = \min \{2, 3, 11, 3\} = 2$$

$$X=\{1,2,4\}, p_2=2$$

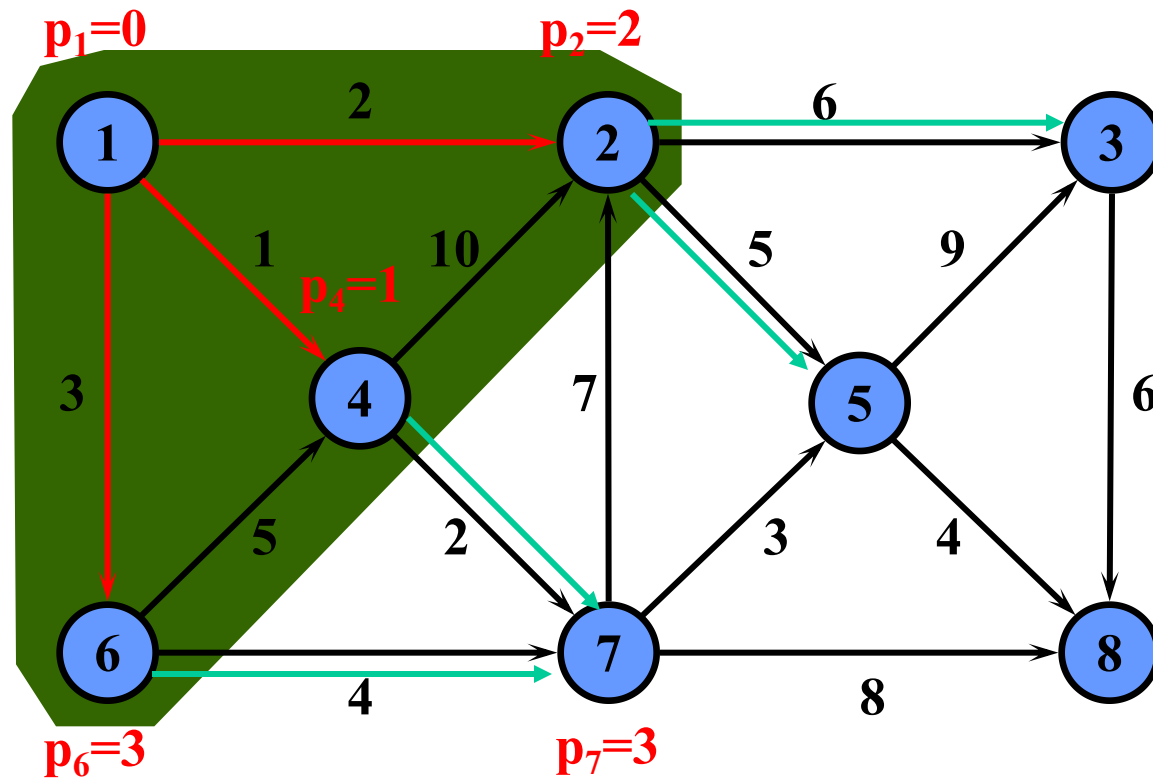
$$X=\{1,2,4\}$$



$$\min \{c_{16}, c_{23}, c_{25}, c_{47}\} = \min \{0+3, 2+6, 2+5, 1+2\} = \min \{3, 8, 7, 3\} = 3$$

$$X=\{1,2,4,6\}, p_6=3$$

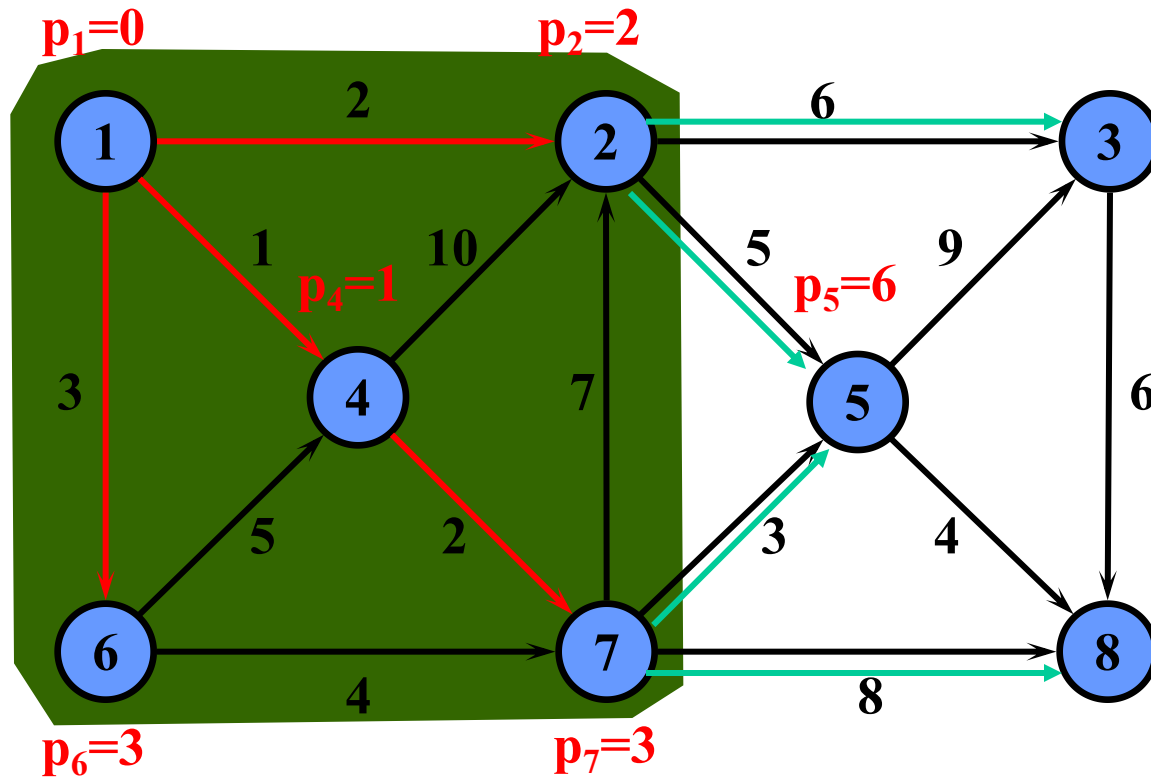
$$X=\{1,2,4,6\}$$



$$\min \{c_{23}, c_{25}, c_{47}, c_{67}\} = \min \{2+6, 2+5, 1+2, 3+4\} = \min \{8, 7, 3, 7\} = 3$$

$$X=\{1,2,4,6,7\}, p_7=3$$

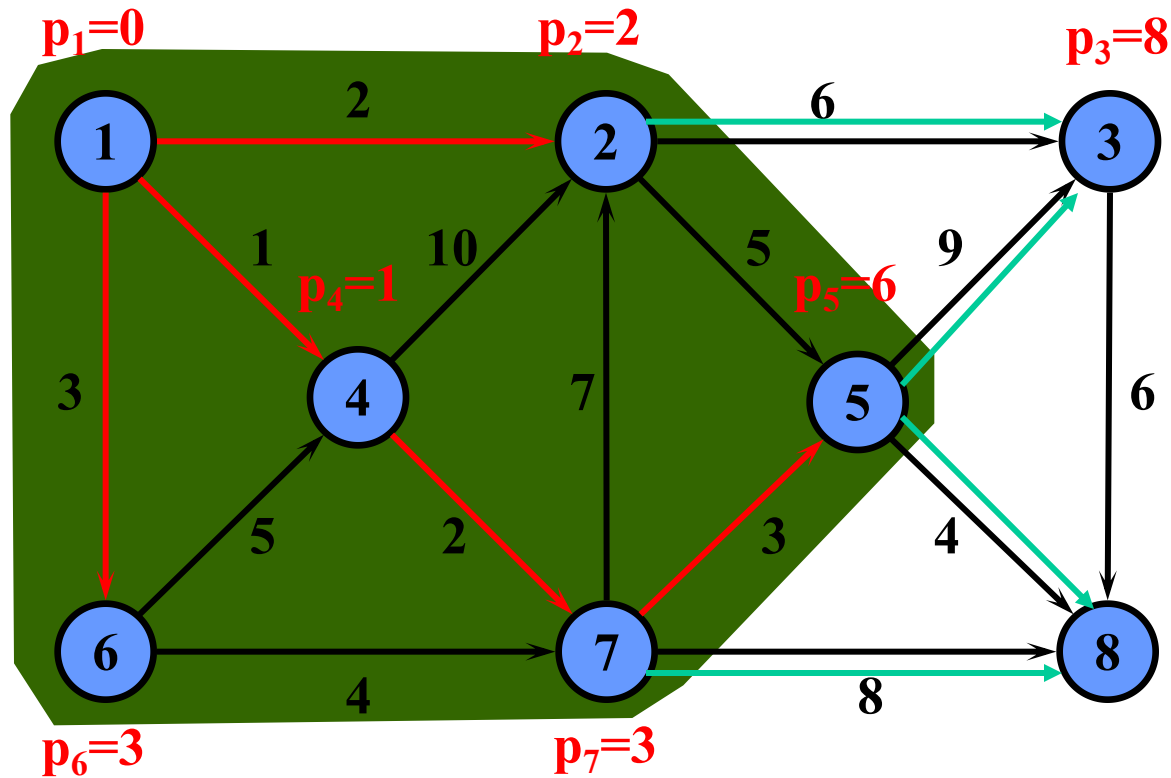
$$X=\{1,2,4,6,7\}$$



$$\min \{c_{23}, c_{25}, c_{75}, c_{78}\} = \min \{2+6, 2+5, 3+3, 3+8\} = \min \{8, 7, 6, 11\} = 6$$

$$X=\{1,2,4,5,6,7\}, p_5=6$$

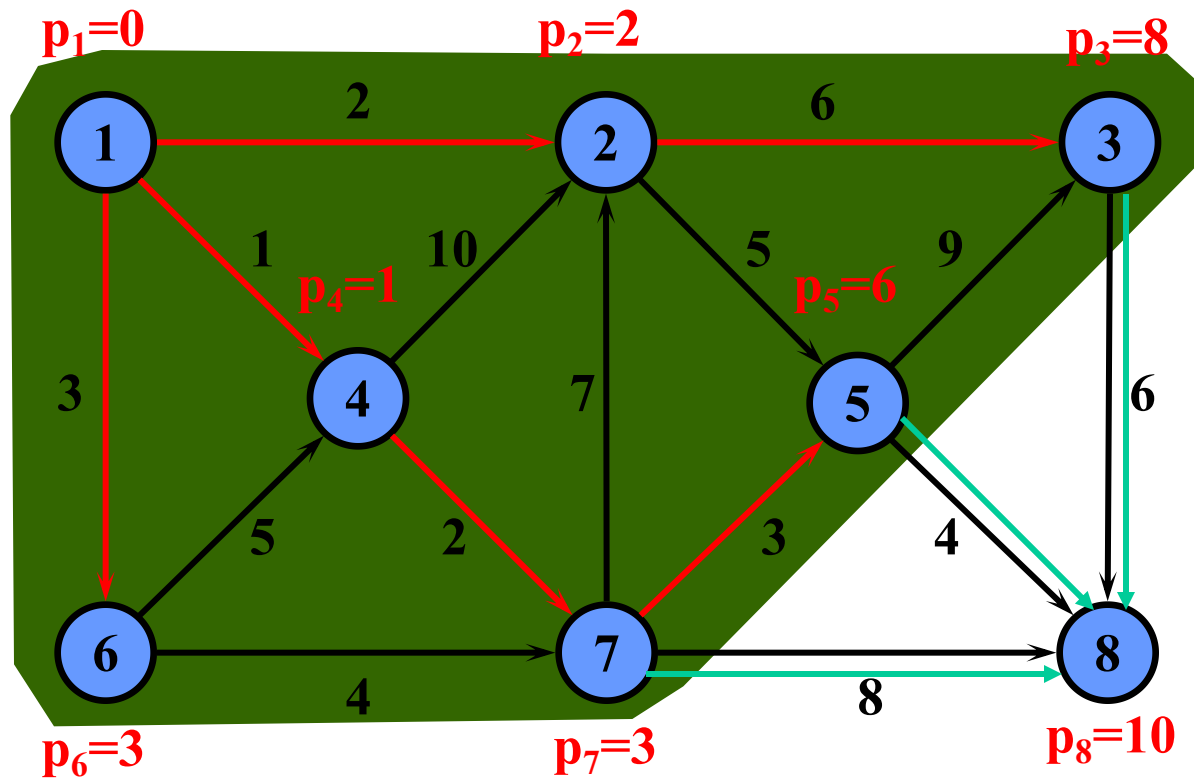
$$X=\{1,2,4,6,7\}$$



$$\min \{c_{23}, c_{53}, c_{58}, c_{78}\} = \min \{2+6, 6+9, 6+4, 3+8\} = \min \{8, 15, 10, 11\} = 8$$

$$X=\{1,2,3,4,5,6,7\}, p_3=8$$

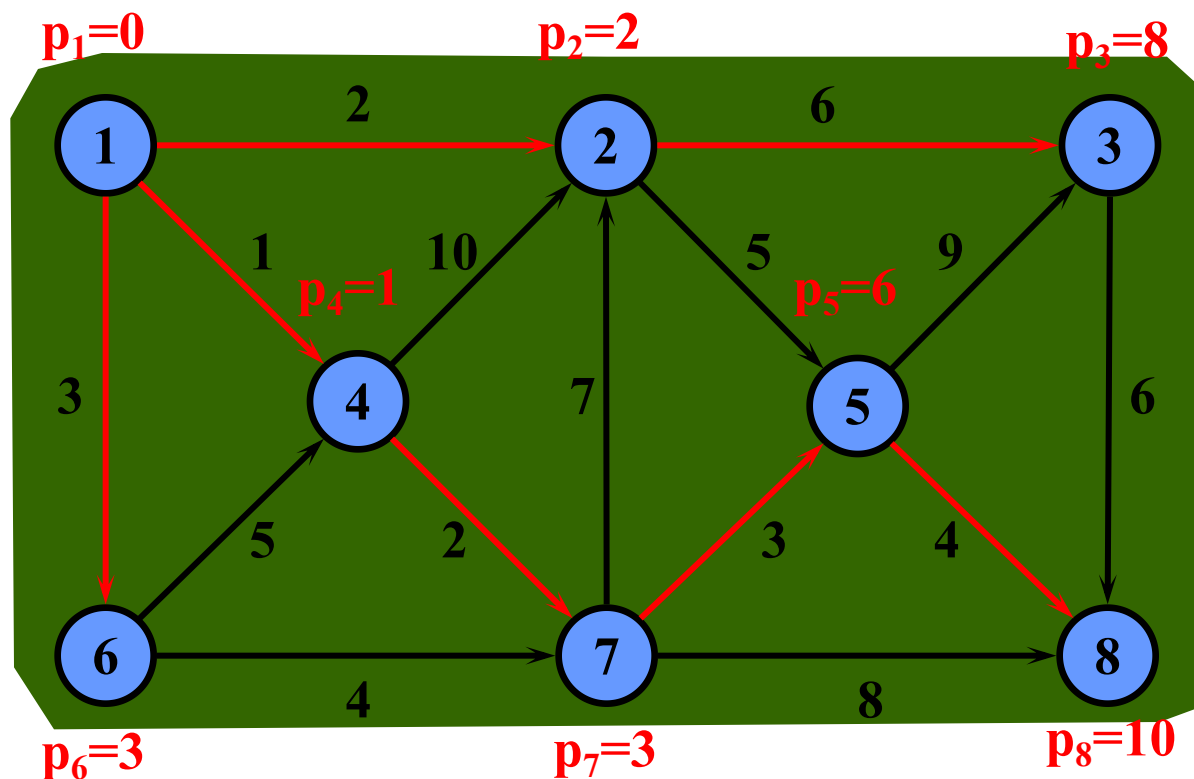
$$X=\{1,2,3,4,6,7\}$$



$$\min \{c_{38}, c_{58}, c_{78}\} = \min \{8+6, 6+4, 3+7\} = \min \{14, 10, 11\} = 10$$

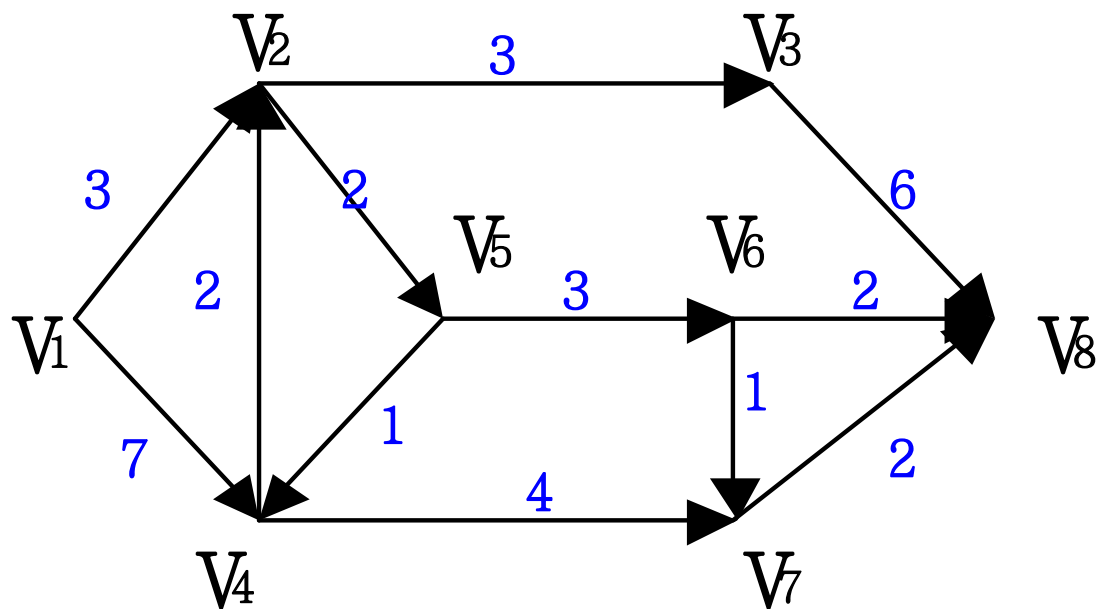
$$X=\{1,2,3,4,5,6,7,8\}, p_8=10$$

$$X=\{1,2,3,4,6,7,8\}$$

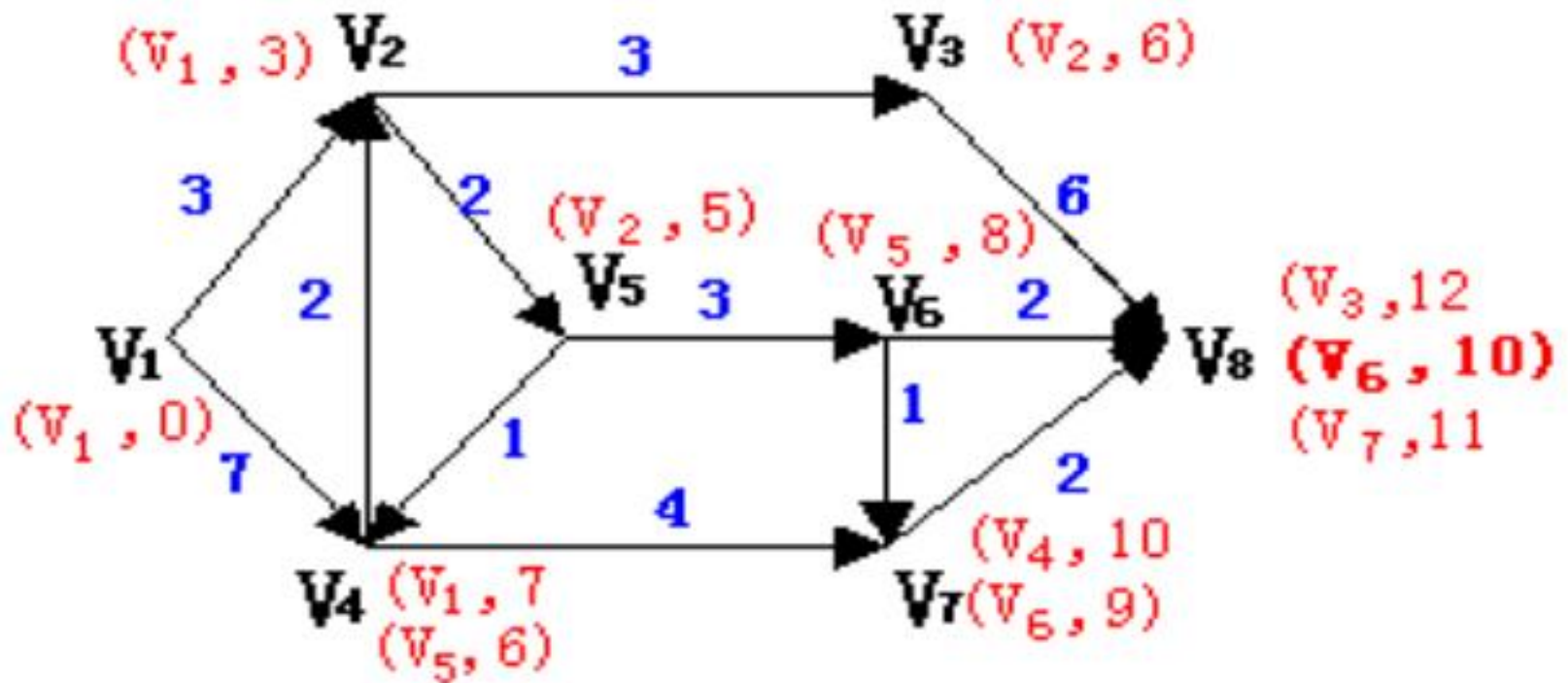


1到8的最短路径为{1, 4, 7, 5, 8}, 长度为10。

求从 V_1 到 V_8 的最短路线。



V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8
① $P=0$	$T=+\infty$	$T=+\infty$	$T=+\infty$	$T=+\infty$	$T=+\infty$	$T=+\infty$	$T=+\infty$
②	$P=T=3$	$T=+\infty$	$T=7$	$T=+\infty$	$T=+\infty$	$T=+\infty$	$T=+\infty$
	③	$T=6$	$T=7$	$P=T=5$	$T=+\infty$	$T=+\infty$	$T=+\infty$
	④	$P=T=6$	$T=6$		$T=8$	$T=+\infty$	$T=+\infty$
		⑤	$P=T=6$		$T=8$	$T=9$	$T=12$
				⑥	$P=T=8$	$T=10$	$T=10$
					⑦	$P=T=9$	$T=11$
再无其它T 标号，所以 $T(V_8)=P(V_8)=10$; $\quad \min L(\mu)=10$						⑧	$P=T=10$



由此看到，此方法不仅求出了从 V_1 到 V_8 的最短路长，同时也求出了从 V_1 到任意一点的最短路长。将从 V_1 到任一点的最短路权标在图上，即可求出从 V_1 到任一点的最短路线。本例中 V_1 到 V_8 的最短路线是：

$$V_1 \rightarrow V_2 \rightarrow V_5 \rightarrow V_6 \rightarrow V_8$$

(二)、逐次逼近法

算法的基本思路与步骤:

首先设任一点 v_i 到任一点 v_j 都有一条弧。

显然, 从 v_1 到 v_j 的最短路是从 v_1 出发, 沿着这条路到某个点 v_i 再沿弧 (v_i, v_j) 到 v_j 。则 v_1 到 v_i 的这条路必然也是 v_1 到 v_i 的所有路中的最短路。设 P_{1j} 表示从 v_1 到 v_j 的最短路长, P_{1i} 表示从 v_1 到 v_i 的最短路长, 则有下列方程:

$$P_{1j} = \min_i \{P_{1i} + l_{ij}\}$$

开始时, 令 $P_{1j}^{(1)} = l_{1j} \quad (j = 1, 2, \dots, n)$

即用 v_1 到 v_j 的直接距离做初始解。

从第二步起, 使用递推公式:

$$P_{1j}^{(k)} = \min_i [P_{1i}^{(k-1)} + l_{ij}] \quad (k = 2, 3, \dots, n)$$

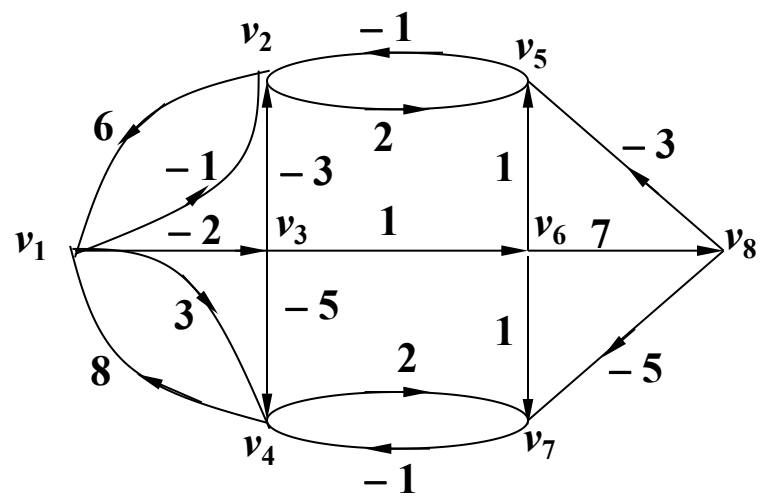
求 $P_{1j}^{(k)}$, 当进行到第 t 步, 若出现

$$P_{1j}^{(t)} = P_{1j}^{(t-1)} \quad (j = 1, 2, \dots, n)$$

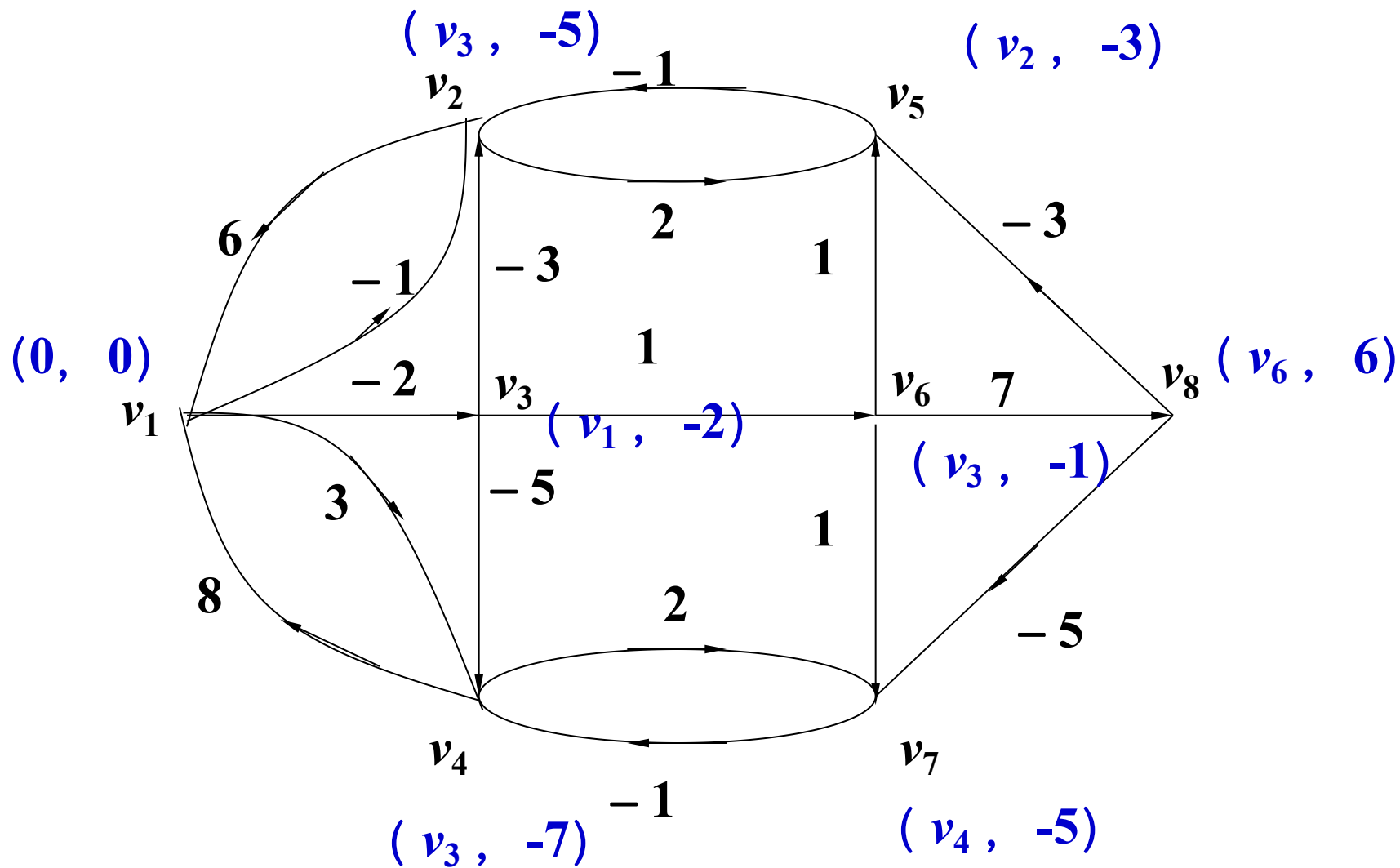
则停止计算, $P_{1j}^{(t)} \quad (j = 1, 2, \dots, n)$ 即为 v_1 到各点的最短路长。

例二、

求图中 v_1 到
各点的最短路



	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	P(1)	P(2)	P(3)	P(4)
v_1	0	-1	-2	3					0	0	0	0
v_2	6	0			2				-1	-5	-5	-5
v_3		-3	0	-5		1			-2	-2	-2	-2
v_4	8			0			2		3	-7	-7	-7
v_5		-1			0					1	-3	-3
v_6					1	0	1	7		-1	-1	-1
v_7				-1			0			5	-5	-5
v_8					-3		-5	0			6	6



(三)、Floyd算法

某些问题中,要求网络上任意两点间的最短路,如例 15 就是这样。这类问题可以用 Dijkstra 算法依次改变起点的办法计算,但比较繁琐。这里介绍的 Floyd 方法(1962 年)可直接求出网络中任意两点间的最短路。

为计算方便,令网络的权矩阵为 $D=(d_{ij})_{n \times n}$, l_{ij} 为 v_i 到 v_j 的距离。

其中

$$d_{ij} = \begin{cases} l_{ij} & \text{当 } (v_i, v_j) \in E \\ \infty & \text{其它} \end{cases}$$

(三)、Floyd算法

算法基本步骤为：

(1) 输入权矩阵 $D^{(0)} = D$ 。

(2) 计算 $D^{(k)} = (d_{ij}^{(k)})_{n \times n} \quad (k=1, 2, 3, \dots, n)$

其中 $d_{ij}^{(k)} = \min[d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}]$

(3) $D^{(n)} = (d_{ij}^{(n)})_{n \times n}$ 中元素 $d_{ij}^{(n)}$ 就是 v_i 到 v_j 的最短路长。

例三、求：5年内，哪些年初购置新设备，使5年内的总费用最小。

解：（1）分析：可行的购置方案（更新计划）是很多的，

如： 1) 每年购置一台新的，则对应的费用为：

$11+11+12+12+13 +5+5+5+5+5 = 84$

2)第一年购置新的，一直用到第五年年底，则总费用

为： $11+5+6+8+11+18 = 59$

显然不同的方案对应不同的费用。

第i年度	1	2	3	4	5
购置费	11	11	12	12	13
设备役龄	0-1	1-2	2-3	3-4	4-5
维修费用	5	6	8	11	18

(2) 方法：将此问题用一个赋权有向图来描述，然后求这个赋权有向图的最短路。

求解步骤：

1) 画赋权有向图：

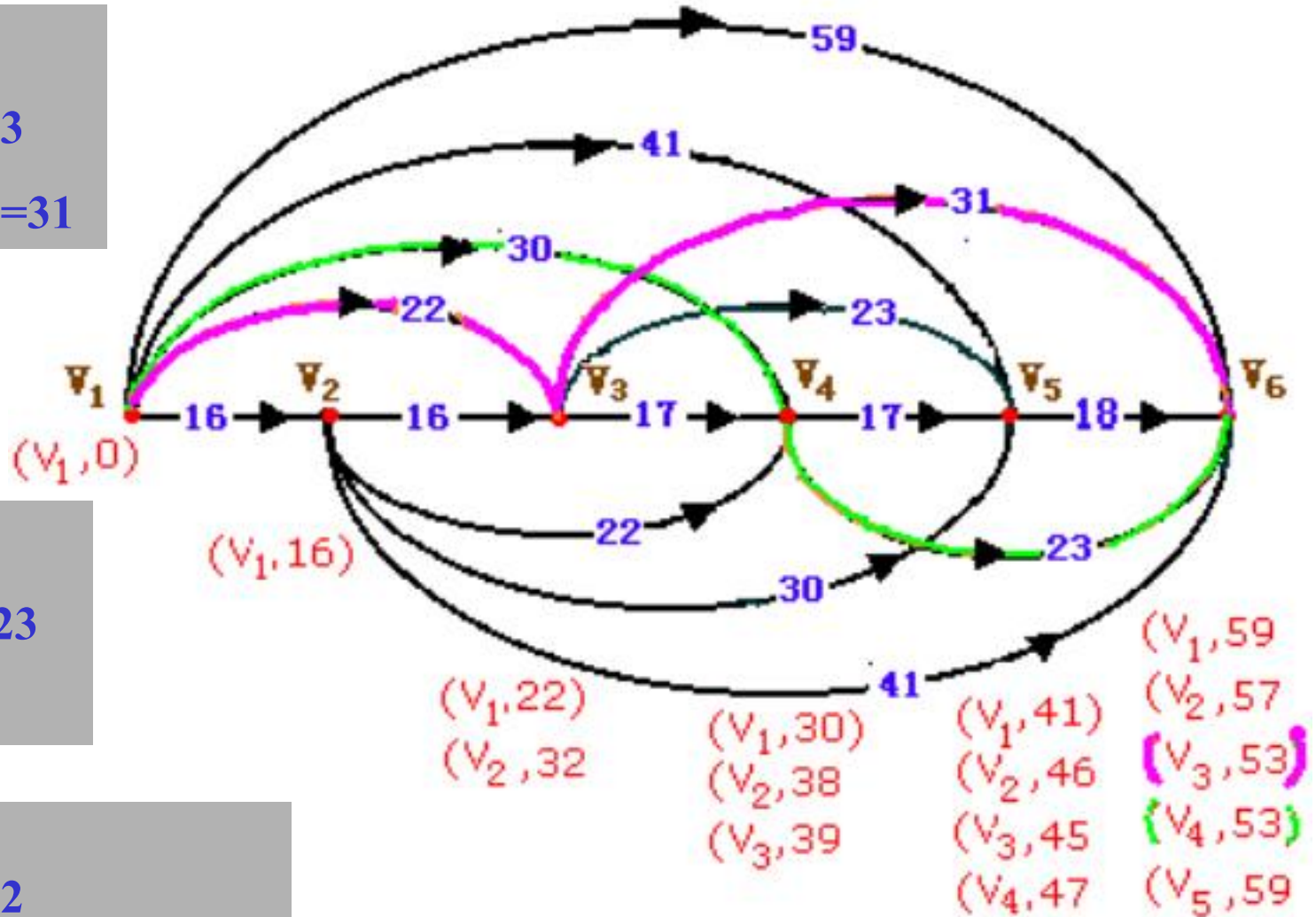
设 V_i 表示第 i 年初购进一台新设备， (V_i, V_j) 表示第 i 年初购买新设备用到第 j 年初 ($j-1$ 年底)，而 W_{ij} 表示相应费用，则 5 年的一个更新计划相当于从 V_1 到 V_6 的一条路。

2) 求解

$$W_{34} = 12 + 5 = 17$$

$$W_{35} = 12 + 5 + 6 = 23$$

$$W_{36} = 12 + 5 + 6 + 8 = 31$$



$$W_{12} = 11 + 5 = 16$$

$$W_{13} = 11 + 5 + 6 = 22$$

$$W_{14} = 11 + 5 + 6 + 8 = 30$$

$$W_{15} = 11 + 5 + 6 + 8 + 11 = 41$$

$$W_{16} = 11 + 5 + 6 + 8 + 11 + 18 = 59$$

$$W_{23} = 11 + 5 = 16$$

$$W_{24} = 11 + 5 + 6 = 22$$

$$W_{25} = 11 + 5 + 6 + 8 = 30$$

$$W_{26} = 11 + 5 + 6 + 8 + 11 = 41$$

例四、 某工厂使用一种设备，这种设备在一定的年限内随着时间的推移逐渐损坏。所以工厂在每年年初都要决定设备是否更新。若购置设备，每年需支付购置费用；若继续使用旧设备，需要支付维修与运行费用，而且随着设备的老化会逐年增加。计划期（五年）内中每年的购置费、维修费与运行费如表所示，工厂要制定今后五年设备更新计划，问采用何种方案才能使包括购置费、维修费与运行费在内的总费用最小。

年份	1	2	3	4	5
购置费	18	20	21	23	24
使用年数	0~1	1~2	2~3	3~4	4~5
维修费	5	7	12	18	25

年份	1	2	3	4	5
购置费	18	20	21	23	24
使用年数	0~1	1~2	2~3	3~4	4~5
维修费	5	7	12	18	25

